HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

MÔN CHUYÊN ĐỀ HỆ THỐNG THÔNG TIN

Chủ đề: Hệ thống tạo câu hỏi trắc nghiệm tự động từ văn bản

Nhóm học phần: 03

Giảng viên: **Phạm Văn Cường**

Nguyễn Thị Ngọc Huyền

Các thành viên nhóm: Đặng Thị Hà - B22DCCN253

Tòng Thị Mai - B22DCCN518

Phạm Văn Sơn - B22DCCN697

Hà Nội, 2026

MỤC LỤC

DANH MỤC HÌNH ẢNH **4**

DANH MỤC CHỮ VIẾT TẮT **5**

MỞ ĐẦU **6**

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG VÀ CÁC CÔNG NGHỆ SỬ DỤNG **7**

1.1. Tổng quan hệ thống 7

1.1.1. Mục tiêu và ý nghĩa 7

1.1.2. Phạm vi 8

1.1.3. Bài toán cần giải quyết 8

1.1.4. Phương hướng giải quyết bài toán 9

1.2. Các công nghệ sử dụng 10

1.2.1. Mô hình tiền huấn luyện ViT5-vinewsqa-qg-ae 10

1.2.2. Nền tảng Ollama và kỹ thuật sinh phương án nhiễu 11

1.2.3. Các công cụ hỗ trợ và giao diện 12

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **14**

2.1. Phân tích chức năng hệ thống 14

2.2. Thiết kế luồng dữ liệu 15

2.3. Thiết kế Prompt cho Ollama và kỹ thuật điều khiển logic 15

2.4. Cơ sở toán học và tối ưu hóa quá trình sinh 16

2.5. Thiết kế giao diện và trải nghiệm người dùng 18

CHƯƠNG 3: CÀI ĐẶT CHƯƠNG TRÌNH **20**

3.1. Cài đặt hệ thống và triển khai hệ thống 20

3.1.1. Môi trường cài đặt và cấu trúc thư mục 20

3.1.2. Triển khai mô hình ViT5-QG-AE 21

3.1.3. Tích hợp Ollama cho distractor generation 22

3.1.4. Xây dựng giao diện Streamlit 23
3.2. Kết quả cài đặt 24
3.3. Kết luận 29
PHẦN KẾT LUẬN 30
1. Tóm tắt các nội dung đã thực hiện 30
2. Những điểm đạt được và hạn chế của hệ thống 30
3. Hướng phát triển trong tương lai 31
TÀI LIỆU THAM KHẢO 32

DANH MỤC HÌNH ẢNH

Hình 1: Sơ đồ luồng xử lý của mô hình shnl/vit5-vinewsqa-qg-ae 11
Hình 2: Giao diện khi người dùng nhập văn bản đầu vào 25
Hình 3: Giao diện chọn số câu hỏi muốn sinh 25
Hình 4: Giao diện kết quả sinh và chỉnh sửa, xáo trộn đáp án của người dùng 26
Hình 5: Giao diện xem trước khi người dùng tải file câu hỏi về 27
Hình 6: Giao diện chọn loại file tải và bảng đáp án 28
Hình 7: Giao diện chọn độ khó và xem lịch sử tạo 28

DANH MỤC CHỮ VIẾT TẮT

Từ viết tắt	Cụm từ đầy đủ	Diễn giải chi tiết trong hệ thống
AI	Artificial Intelligence	Trí tuệ nhân tạo - Công nghệ cốt lõi của hệ thống.
API	Application Programming Interface	Giao diện lập trình ứng dụng - Dùng để kết nối giữa Python và Ollama.
CSV	Comma-Separated Values	Định dạng tệp dữ liệu phân tách bằng dấu phẩy, dùng để xuất bộ đề thi.
GPU	Graphics Processing Unit	Đơn vị xử lý đồ họa, phần cứng giúp tăng tốc độ sinh câu hỏi.
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu nhẹ - Dùng để cấu trúc hóa dữ liệu trắc nghiệm.
LLM	Large Language Model	Mô hình ngôn ngữ lớn - Các mô hình như Llama 3 chạy trên Ollama.
LMS	Learning Management System	Hệ thống quản lý học tập, nơi tiếp nhận dữ liệu trắc nghiệm đầu ra.
MCQ	Multiple Choice Question	Câu hỏi trắc nghiệm nhiều lựa chọn, là sản phẩm cuối cùng của hệ thống.
NER	Named Entity Recognition	Nhận diện thực thể tên riêng - Chức năng trong module spacy_module.py
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên - Lĩnh vực công nghệ mà bài đang áp dụng.
OCR	Optical Character Recognition	Nhận dạng ký tự quang học, hướng phát triển chuyển hình ảnh thành văn bản.
PLM	Pre-trained Language Model	Mô hình ngôn ngữ tiền huấn luyện: Các mô hình nằm trong thư mục plms/
Q&A	Question and Answer	Câu hỏi và đáp án thành phần cơ bản do ViT5 sinh ra.
QG	Question Generation	Sinh câu hỏi tự động - Tác vụ chính của mô hình ViT5.
AE	Answer Extraction	Trích xuất đáp án - Tác vụ đi kèm trong mô hình vit5-vinewsqa-qg-ae.
UI/UX	User Interface / User Experience	Giao diện và trải nghiệm người dùng
ViT5	Vietnamese T5	Biến thể của mô hình T5 (Text-to-Text Transfer Transformer) dành riêng cho tiếng Việt.

MỞ ĐẦU

Trong bối cảnh chuyển đổi số trong giáo dục, việc xây dựng ngân hàng câu hỏi trắc nghiệm từ các nguồn văn bản học thuật ngày càng trở nên cần thiết nhằm hỗ trợ hoạt động giảng dạy và đánh giá. Tuy nhiên, quá trình tạo câu hỏi theo phương pháp thủ công thường tốn nhiều thời gian và công sức của giảng viên. Mặc dù các hệ thống sinh câu hỏi tự động cho tiếng Anh đã phát triển mạnh, các giải pháp dành cho tiếng Việt vẫn còn hạn chế do đặc thù ngôn ngữ và sự thiếu hụt các mô hình chuyên biệt.

Xuất phát từ thực tế đó, đề tài tập trung xây dựng hệ thống tự động tạo câu hỏi trắc nghiệm từ văn bản tiếng Việt bằng cách kết hợp mô hình ViT5 đã được tinh chỉnh và các mô hình ngôn ngữ lớn thông qua Ollama. Trong đó, ViT5 được sử dụng để sinh câu hỏi và đáp án đúng từ ngữ cảnh văn bản, còn các mô hình LLM đảm nhiệm việc tạo các phương án nhiễu hợp lý. Hệ thống được triển khai dưới dạng Web demo nhằm chứng minh tính khả thi của giải pháp và hỗ trợ người dùng nhanh chóng tạo ra các bộ câu hỏi trắc nghiệm chất lượng từ văn bản đầu vào.

CHƯƠNG 1: TỔNG QUAN HỆ THỐNG VÀ CÁC CÔNG NGHỆ SỬ DỤNG

1.1. Tổng quan hệ thống

1.1.1. Mục tiêu và ý nghĩa

Mục tiêu chính của hệ thống là xây dựng một nền tảng thông minh có khả năng tiếp nhận văn bản thô và tự động trích xuất các cặp câu hỏi - đáp án có chất lượng tương đương với việc biên soạn thủ công.

Về mặt kỹ thuật, hệ thống hướng tới việc làm chủ và ứng dụng chuyên sâu các kiến trúc mô hình ngôn ngữ tiên tiến nhất hiện nay, tiêu biểu là ViT5 – một biến thể Transformer được tối ưu hóa riêng cho đặc thù đa thanh điệu và ngữ pháp phức tạp của tiếng Việt. Việc nghiên cứu không chỉ dừng lại ở việc triển khai mô hình có sẵn mà còn đi sâu vào kỹ thuật tinh chỉnh và tối ưu hóa quy trình giải mã để đảm bảo tính tự nhiên của câu hỏi. Bên cạnh đó, việc tích hợp các mô hình ngôn ngữ lớn (LLM) thông qua nền tảng Ollama mở ra khả năng suy luận ngữ nghĩa mạnh mẽ, giúp hệ thống không chỉ bóc tách từ ngữ đơn thuần mà còn hiểu được mối liên hệ logic giữa các thực thể. Đây là bước tiến quan trọng trong việc xây dựng một hệ thống AI có khả năng xử lý ngôn ngữ tiếng Việt một cách tinh tế, giảm thiểu sai sót về ngữ cảnh và nâng cao độ chính xác cho các tác vụ sinh nội dung tự động.

Về mặt thực tiễn, ý nghĩa cốt lõi của hệ thống nằm ở khả năng giải phóng sức lao động, tiết kiệm tối đa thời gian và công sức cho đội ngũ giáo viên, giảng viên cũng như các nhà biên soạn nội dung đào tạo. Trong môi trường giáo dục hiện đại với khối lượng kiến thức khổng lồ, việc phải đọc hiểu thủ công hàng trăm trang tài liệu để đặt từng câu hỏi trắc nghiệm là một thách thức lớn về mặt thời gian. Với hệ thống này, người dùng chỉ cần cung cấp nội dung bài học thô, quy trình "số hóa tri thức" sẽ diễn ra tự động để đưa ra bộ đề thi chuẩn hóa chỉ trong vài giây. Điều này không chỉ giúp nâng cao hiệu suất làm việc gấp nhiều lần mà còn đảm bảo tính khách quan, đa dạng trong việc đánh giá tri thức, loại bỏ sự chủ quan hoặc sai sót do mệt mỏi của con người, từ đó góp phần thúc đẩy quá trình chuyển đổi số giáo dục một cách thực chất và hiệu quả.

1.1.2. Phạm vi

Hệ thống tập trung xử lý các văn bản tiếng Việt thuộc nhiều lĩnh vực khác nhau như tin tức, giáo khoa, tài liệu học thuật và văn bản hành chính. Phạm vi kỹ thuật bao gồm:

- Đầu vào: Văn bản phi cấu trúc định dạng thuần túy.
- Xử lý: Nhận diện thực thể, sinh câu hỏi và sinh phương án nhiễu.
- Đầu ra: Các bộ câu hỏi trắc nghiệm hoàn chỉnh gồm 4 lựa chọn (A, B, C, D) với một đáp án đúng duy nhất, có khả năng xuất ra các định dạng chuẩn như CSV hoặc JSON.

1.1.3. Bài toán cần giải quyết

Hệ thống được thiết kế để đối đầu và giải quyết ba thách thức cốt lõi, mang tính quyết định trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP) khi ứng dụng vào môi trường giáo dục chuyên biệt. Đây là những điểm nghẽn mà các phương pháp truyền thống dựa trên quy tắc thường gặp thất bại:

- Bài toán trích xuất thực thể và chọn lọc thông tin quan trọng:

  Trong một văn bản dài với mật độ thông tin dày đặc, việc xác định đâu là nội dung xứng đáng để đặt câu hỏi là một bài toán phân loại phức tạp. Hệ thống phải có khả năng phân tích ngữ cảnh để nhận diện các thực thể quan trọng như địa danh, con số, nhân vật lịch sử hay các thuật ngữ chuyên ngành. Thách thức ở đây không chỉ dừng lại ở việc nhận diện (NER) mà còn là đánh giá tầm quan trọng của thực thể đó trong toàn bộ văn bản.

  Nếu không giải quyết tốt bài toán này, hệ thống sẽ sinh ra những câu hỏi vụn vặt, thiếu trọng tâm hoặc bỏ lỡ những kiến thức cốt lõi mà người học cần nắm vững.
- Bài toán logic ngữ nghĩa và tính toàn vẹn thông tin:

  Đối với tiếng Việt - một ngôn ngữ có cấu trúc ngữ pháp linh hoạt và phong phú về mặt thanh điệu, việc sinh câu hỏi tự động đòi hỏi độ chính xác cực cao về mặt cú pháp. Bài toán đặt ra là làm thế nào để câu hỏi sinh ra không chỉ đúng về

mặt ngữ pháp mà còn phải duy trì tính trung thực tuyệt đối so với văn bản gốc. Hệ thống phải tránh được hiện tượng "ảo giác" (hallucination) - một lỗi phổ biến của các mô hình AI khi tự ý thêm thắt các thông tin không có thực. Việc đảm bảo logic ngữ nghĩa giúp câu hỏi trở nên mạch lạc, chuyên nghiệp và phản ánh đúng giá trị tri thức của tài liệu học thuật.

- Bài toán sinh phương án nhiễu:

  Đây được coi là thách thức lớn nhất và là "linh hồn" của một câu hỏi trắc nghiệm chất lượng. Một câu hỏi hay không chỉ nằm ở câu hỏi đúng mà còn ở các lựa chọn sai có tính thuyết phục cao. Bài toán yêu cầu hệ thống phải tạo ra được các phương án nhiễu đáp ứng tiêu chuẩn kép: chúng phải sai về mặt sự thật dựa trên văn bản, nhưng đồng thời phải "có vẻ đúng" về mặt logic và đặc biệt phải cùng loại thực thể với đáp án chính xác.

  Ví dụ, nếu đáp án đúng là một vị vua, các phương án nhiễu cũng phải là những vị vua trong cùng thời đại. Việc tạo ra độ nhiễu hợp lý sẽ ngăn chặn việc người học dùng phương pháp loại trừ mẹo, buộc họ phải thực sự tư duy và thấu hiểu nội dung để tìm ra đáp án, từ đó đảm bảo tính phân loại và giá trị đánh giá của đề thi.

1.1.4. Phương hướng giải quyết bài toán

Để giải quyết triệt để các thách thức về ngôn ngữ và logic trong việc sinh câu hỏi trắc nghiệm, hệ thống Sinh câu hỏi trắc nghiệm tự động từ văn bản triển khai một quy trình phối hợp đa mô hình, tận dụng thế mạnh riêng biệt của từng kiến trúc AI từ chuyên biệt đến tổng quát. Phương hướng giải quyết được cụ thể hóa qua ba giai đoạn chiến lược.

Tận dụng sức mạnh của mô hình Sequence-to-Sequence chuyên biệt (ViT5). Để giải quyết bài toán sinh câu hỏi và trích xuất đáp án, hệ thống lựa chọn mô hình ViT5 làm thành phần cốt lõi. Khác với các mô hình ngôn ngữ lớn đa năng, ViT5 được huấn luyện tập trung vào các tác vụ chuyển đổi văn bản sang văn bản tại thị trường Việt Nam. Phương hướng cụ thể là sử dụng kiến trúc Encoder-Decoder để nén ngữ cảnh văn bản và giải mã thành các cặp câu hỏi - đáp án (Q&A). Việc sử dụng một mô hình chuyên

biệt giúp hệ thống kiểm soát chặt chẽ tính chính xác của thông tin, tránh hiện tượng lan man và đảm bảo câu hỏi luôn bám sát vào các tình tiết có thực trong văn bản đầu vào.

Tiếp theo là ứng dụng mô hình ngôn ngữ lớn (LLM) và Prompt Engineering. Đối với bài toán sinh phương án nhiễu vốn đòi hỏi sự sáng tạo và khả năng suy luận logic, hệ thống chuyển hướng sử dụng các LLM thông qua nền tảng Ollama. Thay vì dùng các thuật toán so khớp từ vựng đơn giản, hệ thống áp dụng kỹ thuật Prompt Engineering để điều hướng mô hình. Bằng cách cung cấp ngữ cảnh bao gồm "Câu hỏi" và "Đáp án đúng", hệ thống yêu cầu LLM thực hiện nhiệm vụ suy luận để tìm ra các thực thể có tính chất tương đồng nhưng mang giá trị sai lệch. Đây là phương pháp tối ưu để tạo ra các lựa chọn sai có độ nhiễu cao, khiến người học không thể dễ dàng vượt qua bằng các mẹo ngôn ngữ thông thường.

Tối ưu hóa quy trình tương tác người máy, ệ thống không vận hành theo cơ chế "hộp đen" hoàn toàn mà hướng tới sự phối hợp giữa trí tuệ nhân tạo và chuyên môn của người dùng. Thông qua giao diện Dashboard được xây dựng bằng Streamlit, phương hướng giải quyết bài toán thực tế là cung cấp một quy trình kiểm duyệt hai lớp. Lớp thứ nhất do AI thực hiện để tạo ra bản thảo sơ bộ, lớp thứ hai cho phép giáo viên tinh chỉnh, sửa đổi hoặc loại bỏ các câu hỏi chưa đạt yêu cầu. Sự kết hợp này đảm bảo rằng sản phẩm cuối cùng luôn đạt tiêu chuẩn sư phạm cao nhất, đồng thời vẫn giữ được tốc độ xử lý vượt trội của máy móc.

1.2. Các công nghệ sử dụng

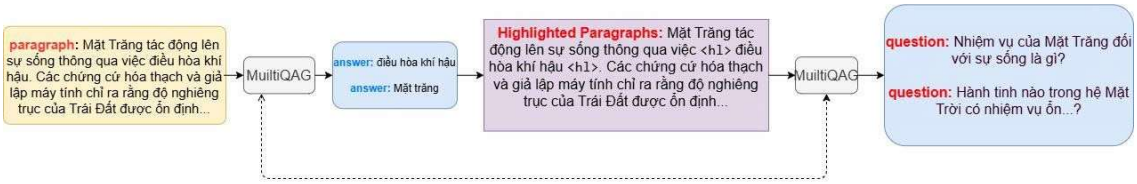
1.2.1. Mô hình tiền huấn luyện ViT5-vinewsqa-qg-ae

Hệ thống sử dụng mô hình mã nguồn mở *shnl/vit5-vinewsqa-qg-ae* dựa trên kiến trúc T5 (Text-to-Text Transfer Transformer). Đây là kiến trúc Seq2Seq mạnh mẽ xử lý mọi bài toán NLP dưới dạng văn bản sang văn bản (Text-to-Text). Cơ chế hoạt động: Mô hình nhận đầu vào là đoạn văn bản và sử dụng các chỉ dẫn cụ thể để trích xuất ra cặp câu hỏi và đáp án. Công thức toán học: Mô hình hoạt động dựa trên việc cực đại hóa xác suất có điều kiện của chuỗi đầu ra Y (gồm câu hỏi và đáp án) khi biết chuỗi đầu vào X (Ngữ cảnh):

$$P(Y|X) = \prod_{t=1}^{n} P(y_t|y_{<t}, X)$$

Trong đó, việc sử dụng mô hình pre-trained giúp chúng ta thừa hưởng bộ tham số θ đã được tối ưu hóa trên các tập dữ liệu lớn như ViQuAD, giúp câu hỏi sinh ra tự nhiên và chính xác về mặt ngữ nghĩa.

Bên cạnh khả năng hiểu ngữ cảnh sâu sắc, mô hình *shnl/vit5-vinewsqa-qg-ae* còn tối ưu hóa quá trình sinh văn bản thông qua cơ chế Beam Search trong giai đoạn giải mã. Thay vì chỉ chọn từ có xác suất cao nhất tại mỗi bước, Beam Search duy trì một số lượng k các chuỗi từ tiềm năng (gọi là beam width), giúp giảm thiểu sai lầm cục bộ và đảm bảo câu hỏi sinh ra có cấu trúc cú pháp hoàn chỉnh, tự nhiên như ngôn ngữ người dùng. Việc sử dụng một mô hình đã được tinh chỉnh sẵn trên tập dữ liệu chuyên biệt về câu hỏi - đáp án tiếng Việt giúp hệ thống vượt qua được rào cản về sự đa nghĩa và cấu trúc câu phức tạp của tiếng Việt mà các mô hình tổng quát thường gặp phải. Điều này không chỉ bảo đảm tính chính xác về mặt nội dung so với văn bản gốc mà còn giúp tiết kiệm đáng kể tài nguyên tính toán, cho phép hệ thống vận hành ổn định và phản hồi nhanh chóng trong môi trường thực tế.



Hình 1: Sơ đồ luồng xử lý của mô hình shnl/vit5-vinewsqa-qg-ae

1.2.2. Nền tảng Ollama và kỹ thuật sinh phương án nhiễu

Trong kiến trúc tổng thể của hệ thống, Ollama đóng vai trò là một "engine" thực thi các mô hình ngôn ngữ lớn (LLM) cục bộ, cung cấp khả năng tư duy ngôn ngữ linh hoạt để hoàn thiện các thành phần còn lại của một câu hỏi trắc nghiệm sau khi ViT5 đã xác định được câu hỏi và đáp án đúng. Thay vì sử dụng các phương pháp tìm kiếm từ đồng nghĩa hay dựa trên khoảng cách từ vựng truyền thống thường dẫn đến kết quả khô cứng, Ollama cho phép hệ thống khai thác sức mạnh của các mô hình như Llama 3 hoặc PhoGPT để thực hiện tác vụ sinh phương án nhiễu một cách thông minh thông qua cơ chế Prompt Engineering. Bằng cách gửi một chỉ dẫn ngữ cảnh chi tiết bao gồm nội dung câu hỏi và đáp án chính xác, hệ thống yêu cầu Ollama tạo ra ba phương án sai nhưng

phải đảm bảo tính tương đồng về mặt thực thể và ngữ nghĩa với đáp án đúng, tạo nên sự đánh đố cần thiết cho một bài thi trắc nghiệm.

Cơ chế này hoạt động dựa trên khả năng suy luận xác suất của LLM, trong đó mô hình sẽ tìm kiếm các token có độ tương quan cao với chủ đề đang thảo luận nhưng lại sai lệch về mặt sự thật so với ngữ cảnh đã cho. Việc chạy Ollama cục bộ giúp hệ thống duy trì quyền kiểm soát dữ liệu hoàn toàn, không phụ thuộc vào các API bên ngoài, đồng thời giảm thiểu độ trễ trong quá trình phản hồi giữa các thành phần của pipeline.

Điểm ưu việt của việc tích hợp Ollama nằm ở khả năng điều chỉnh độ khó của phương án nhiễu thông qua việc tinh chỉnh câu lệnh, giúp các lựa chọn sai không chỉ đơn thuần là những từ ngữ ngẫu nhiên mà là các khái niệm có sự liên quan logic, từ đó buộc người học phải có sự phân tích sâu sắc mới có thể chọn được đáp án chính xác nhất. Sự kết hợp này tạo nên một quy trình khép kín, nơi ViT5 đảm nhiệm tính chính xác dựa trên dữ liệu chuyên biệt và Ollama đảm nhiệm tính sáng tạo và đa dạng dựa trên tri thức rộng lớn của các mô hình ngôn ngữ khổng lồ.

1.2.3. Các công cụ hỗ trợ và giao diện

- Hugging Face Transformers: Đây là thư viện mã nguồn mở đóng vai trò "cầu nối" kỹ thuật quan trọng nhất, cho phép hệ thống tương tác trực tiếp với kho lưu trữ mô hình khổng lồ toàn cầu. Trong dự án, thư viện này được sử dụng để tải cấu hình, trọng số mô hình (weights) và bộ mã hóa (tokenizer) của *shnl/vit5-vinewsqa-qg-ae*
- SpaCy: Đóng vai trò là bộ khung xử lý tiền ngôn ngữ chuyên sâu trước khi dữ liệu được đưa vào mô hình AI. Hệ thống tận dụng SpaCy để thực hiện các tác vụ như phân đoạn văn bản, gán nhãn từ loại và đặc biệt là nhận diện thực thể tên riêng. Việc xác định chính xác các thực thể như tên người, địa danh, tổ chức hay mốc thời gian giúp hệ thống định vị được các trọng tâm thông tin. Từ đó, mô hình ViT5 có thể tập trung vào các từ khóa này để sinh ra những câu hỏi có ý nghĩa, tránh tình trạng đặt câu hỏi vào các chi tiết vụn vặt hoặc không quan trọng trong văn bản

- Streamlit: Đây là framework hiện đại được sử dụng để xây dựng giao diện người dùng và logic điều hướng cho ứng dụng web. Với Streamlit, dự án đã chuyển đổi từ một kịch bản code thô thành một bảng điều khiển chuyên nghiệp dành cho người dùng. Giao diện này không chỉ cung cấp không gian để nhập văn bản ngữ cảnh mà còn tích hợp các bảng tương tác thời gian thực, cho phép người dùng xem trước, chỉnh sửa nội dung câu hỏi do ViT5 sinh ra, và tùy chỉnh các phương án nhiễu từ Ollama. Cuối cùng, Streamlit hỗ trợ các module xuất dữ liệu giúp người dùng tải bộ đề thi về để sử dụng ngay.

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích chức năng hệ thống

Hệ thống được thiết kế để giải quyết bài toán chuyển đổi văn bản phi cấu trúc thành cấu trúc trắc nghiệm chuẩn hóa. Các chức năng chính bao gồm:

Phân tích ngữ cảnh và tiền xử lý văn bản là chức năng cửa ngõ của hệ thống, chịu trách nhiệm tiếp nhận và làm sạch các nguồn văn bản phi cấu trúc từ người dùng. Hệ thống không chỉ đơn thuần đọc văn bản mà còn thực hiện các kỹ thuật NLP chuyên sâu như tách câu, chuẩn hóa từ vựng và nhận diện thực thể tên riêng (NER). Việc bóc tách các thực thể như địa danh, nhân vật, mốc thời gian hay các khái niệm then chốt giúp hệ thống xác định được những vùng tri thức quan trọng. Qua đó, hệ thống có thể giới hạn phạm vi khai thác thông tin, tránh việc sinh câu hỏi lan man và đảm bảo rằng mọi câu hỏi đều tập trung vào những nội dung cốt lõi nhất của tài liệu.

Sinh cặp Câu hỏi - Đáp án là chức năng trung tâm, sử dụng sức mạnh của mô hình chuyên biệt *shnl/vit5-vinewsqa-qg-ae*. Thay vì dựa trên các quy tắc ngữ pháp cứng nhắc, mô hình này phân tích mối quan hệ ngữ nghĩa giữa các thành phần trong câu để tự động trích xuất đáp án đúng và đặt câu hỏi tương ứng. Chức năng này đảm bảo tính trung thực của dữ liệu, nghĩa là câu hỏi sinh ra luôn có căn cứ xác thực từ văn bản gốc. Việc sử dụng mô hình đã được tinh chỉnh giúp câu hỏi có văn phong tự nhiên, đa dạng về hình thức (câu hỏi Ai, Cái gì, Ở đâu, Tại sao) và bám sát logic của nội dung học thuật.

Sáng tạo phương án sai, chức năng này đóng vai trò quyết định đến độ khó và chất lượng sư phạm của một câu hỏi trắc nghiệm. Bằng cách gửi yêu cầu đến LLM thông qua engine Ollama, hệ thống thực hiện quá trình suy luận để tạo ra các phương án nhiễu có tính thuyết phục cao. Các lựa chọn sai này được thiết kế dựa trên ngữ cảnh của câu hỏi và đáp án đúng, đảm bảo chúng có sự tương đồng về loại thực thể hoặc logic nhưng lại sai lệch về mặt thông tin thực tế. Điều này giúp loại bỏ các phương án "rác" hoặc quá lộ liễu, buộc người học phải có sự phân tích kỹ lưỡng và hiểu sâu bài học mới có thể chọn được đáp án chính xác.

Quản trị, Hiệu chỉnh và Xuất bản: Để đảm bảo hệ thống không chỉ là một công cụ AI thuần túy mà còn là một trợ thủ đắc lực cho giáo viên, chức năng này cung cấp một quy trình kiểm soát chất lượng. Người dùng có quyền can thiệp vào kết quả của AI: chỉnh sửa câu chữ, thay đổi đáp án hoặc yêu cầu sinh lại các phương án nhiễu nếu chưa ưng ý. Sau khi hoàn thiện, hệ thống hỗ trợ xuất dữ liệu ra các cấu trúc chuẩn hóa (như CSV, JSON hoặc định dạng file văn bản), giúp dễ dàng tích hợp vào các hệ thống quản lý học tập (LMS) hoặc in ấn trực tiếp phục vụ cho việc kiểm tra, đánh giá trong môi trường giáo dục.

2.2. Thiết kế luồng dữ liệu

Hệ thống vận hành theo một luồng dữ liệu khép kín, trong đó đầu ra của module này là đầu vào của module kế tiếp:

1. Input Layer: Người dùng nhập đoạn văn bản thông qua giao diện Streamlit.
2. Processing Layer 1 (ViT5): Hệ thống kết hợp văn bản gốc với các chỉ dẫn chuyên biệt. Luồng dữ liệu lúc này đi qua cơ chế Encoder-Decoder của ViT5: Encoder mã hóa ngữ cảnh thành các vector biểu diễn không gian, và Decoder thực hiện giải mã tự hồi quy để sinh ra cặp câu hỏi và đáp án đúng. Kết quả đầu ra của phân lớp này là một danh sách các cặp $(Q, A_{correct})$ có độ chính xác cao về mặt nội dung thực tế.
3. Processing Layer 2 (Ollama): hệ thống thực hiện đóng gói dữ liệu vào một cấu trúc Prompt chuyên biệt. Yêu cầu này được gửi đến engine Ollama thông qua giao thức API nội bộ. Ollama tiếp nhận cặp $(Q, A_{correct})$, thực hiện suy luận dựa trên tri thức rộng lớn của LLM để sinh ra các phương án nhiễu $(D_1, D_2, D_3, \dots)$
4. Output Layer: Hệ thống tổng hợp thành bộ MCQ hoàn chỉnh và hiển thị lên giao diện.

2.3. Thiết kế Prompt cho Ollama và kỹ thuật điều khiển logic

Đây là giai đoạn then chốt quyết định tính thông minh và giá trị sư phạm của toàn hệ thống, nơi sức mạnh của các mô hình ngôn ngữ lớn (LLM) được khai thác tối đa thông qua kỹ thuật Instructional Prompting. Thay vì để Ollama tự do sinh văn bản một cách ngẫu hứng, hệ thống thực hiện điều khiển mô hình bằng một cấu trúc chỉ dẫn chặt

chẽ, bắt đầu từ việc thiết lập vai trò cụ thể cho AI như một chuyên gia khảo thí dày dạn kinh nghiệm. Việc xác định vai trò giúp mô hình tập trung vào ngôn ngữ học thuật, đảm bảo các phương án được tạo ra mang tính chuyên môn cao thay vì các câu trả lời giao tiếp thông thường. Ngay sau đó, hệ thống cung cấp ngữ cảnh chi tiết bao gồm câu hỏi và đáp án chính xác đã được ViT5 trích xuất, giúp mô hình hiểu rõ phạm vi tri thức và xác định chính xác thực thể cần xử lý.

Để đảm bảo chất lượng cho các câu hỏi trắc nghiệm, hệ thống áp dụng một tập hợp các ràng buộc logic nghiêm ngặt trong quá trình cấu trúc hóa Prompt. Ràng buộc quan trọng nhất là tính đồng nhất thực thể (Semantic Type Consistency), yêu cầu các phương án nhiễu phải thuộc cùng một nhóm khái niệm với đáp án đúng; chẳng hạn, nếu đáp án đúng là một danh nhân lịch sử thì các lựa chọn sai cũng phải là những nhân vật lịch sử có liên quan. Bên cạnh đó, các phương án nhiễu được yêu cầu phải mang tính hợp lý giả định, tức là chúng phải sai về mặt nội dung dựa trên văn bản gốc nhưng lại có vẻ đúng về mặt logic tổng quát để thử thách khả năng phân tích của người học. Cuối cùng, hệ thống áp đặt một định dạng đầu ra chuẩn hóa về mặt kỹ thuật, yêu cầu Ollama trả về kết quả dưới dạng danh sách ngăn cách bởi các ký hiệu đặc biệt để mã nguồn Python có thể bóc tách dữ liệu một cách tự động và hiển thị lên giao diện mà không gặp lỗi cấu trúc.

Ví dụ cụ thể về quá trình này có thể thấy qua việc xử lý một câu hỏi về thủ đô Việt Nam. Khi ViT5 cung cấp câu hỏi "Thành phố nào là thủ đô của Việt Nam?" với đáp án đúng là "Hà Nội", hệ thống sẽ gửi một Prompt yêu cầu Ollama đóng vai trò chuyên gia biên soạn đề thi, đưa ra đúng ba tên thành phố lớn khác của Việt Nam làm phương án nhiễu, đồng thời yêu cầu không giải thích thêm để giữ định dạng đầu ra sạch. Kết quả là Ollama sẽ trả về các lựa chọn như "TP. Hồ Chí Minh, Đà Nẵng, Hải Phòng". Toàn bộ chuỗi dữ liệu này sau đó được hệ thống tổng hợp thành một câu hỏi trắc nghiệm hoàn chỉnh gồm bốn phương án A, B, C, D, tạo nên một bài kiểm tra có độ nhiễu cao và đảm bảo tính khách quan trong việc đánh giá năng lực người học.

2.4. Cơ sở toán học và tối ưu hóa quá trình sinh

Để tối ưu hóa tính logic và độ khó của các lựa chọn, hệ thống không chỉ dựa vào các chỉ dẫn văn bản thuần túy mà còn kiểm soát quá trình sinh của Ollama thông qua

các tham số xác suất và hình học không gian ngôn ngữ. Đầu tiên, việc điều chỉnh sự cân bằng giữa tính sáng tạo và tính chính xác của phương án nhiễu được thực hiện thông qua tham số nhiệt độ (Temperature - T). Xác suất để một từ y_i được lựa chọn tại mỗi bước sinh được tính toán dựa trên hàm Softmax đã được điều chỉnh:

$$P(y_i) = \frac{\exp(z_i/T)}{\sum_j^{\exp} \quad (z_j/T)}$$

Trong đó: z_i là giá trị logit của token đó. Bằng việc thiết lập T ở mức phù hợp (thường từ 0.7 đến 0.8), hệ thống đảm bảo Ollama không chọn các từ quá hiển nhiên (khi T thấp) nhưng cũng không sinh ra các nội dung rời rạc, thiếu liên kết (khi T quá cao). Điều này giúp các phương án nhiễu duy trì được sự tự nhiên và gây nhiễu tốt cho người học.

Bên cạnh đó, để hiện thực hóa ràng buộc về tính đồng nhất thực thể, hệ thống hướng tới việc tối ưu hóa độ tương đồng ngữ nghĩa giữa đáp án đúng (A) và các phương án nhiễu (D). Về mặt kỹ thuật, mỗi từ ngữ đều được biểu diễn dưới dạng các vector trong không gian n-chiều. Độ tương đồng này được định nghĩa thông qua công thức Cosine Similarity:

$$similarity(A, D) = \frac{V_A . V_D}{||V_A||||V_D||}$$

Mục tiêu của thiết kế Prompt là điều hướng mô hình chọn ra các phương án D sao cho giá trị tương đồng này nằm trong khoảng tối ưu. Nếu độ tương đồng quá cao, phương án nhiễu có nguy cơ trở thành từ đồng nghĩa với đáp án đúng; nếu quá thấp, phương án sẽ trở nên quá xa lạ và dễ bị loại trừ. Cuối cùng, toàn bộ quá trình sinh danh sách phương án nhiễu D* có thể được xem là một bài toán tối ưu hóa xác suất có điều kiện dựa trên Prompt chỉ dẫn (P), câu hỏi (Q) và đáp án chính xác (A):

$$D^* = \arg\max_D P(\, D \mid P, Q, A \,)$$

Việc áp dụng các cơ sở toán học này vào thiết kế giúp hệ thống không chỉ dừng lại ở việc sinh văn bản ngẫu nhiên mà thực sự trở thành một công cụ khảo thí có độ tin cậy và tính khoa học cao.

2.5. Thiết kế giao diện và trải nghiệm người dùng

Giao diện của hệ thống tự động tạo câu hỏi trắc nghiệm từ văn bản được xây dựng trên nền tảng Streamlit, một framework hiện đại cho phép chuyển đổi các mô hình Python thành ứng dụng web tương tác một cách nhanh chóng. Mục tiêu cốt lõi của thiết kế là tối giản hóa sự phức tạp kỹ thuật của các mô hình AI phía sau (ViT5 và Ollama), cung cấp cho người dùng một quy trình làm việc trực quan, liền mạch. Giao diện được phân chia thành ba khu vực chức năng chính nhằm tối ưu hóa trải nghiệm người dùng:

- Khu vực điều khiển và nhập liệu: Đây là điểm khởi đầu của quy trình, nơi người dùng cung cấp nguyên liệu cho hệ thống. Thay vì chỉ là một ô nhập văn bản đơn thuần, khu vực này được thiết kế để hỗ trợ xử lý các đoạn văn bản dài thông qua thành phần st.text_area. Tại đây, người dùng có thể linh hoạt điều chỉnh tham số thực thi số lượng câu hỏi mong muốn.
- Khu vực xử lý thời gian thực và phản hồi trạng thái: Do quá trình thực thi của mô hình ViT5 và đặc biệt là việc sinh phương án nhiễu từ Ollama tiêu tốn tài nguyên tính toán nhất định, hệ thống tích hợp các thanh trạng thái (st.progress) và thông báo động (st.spinner). Điều này giúp giải quyết bài toán tâm lý người dùng, cho họ biết hệ thống đang vận hành ở giai đoạn nào (đang phân tích văn bản, đang sinh câu hỏi hay đang tạo phương án nhiễu). Việc cung cấp phản hồi tức thì giúp giảm thiểu sự không chắc chắn và tăng tính minh bạch cho quy trình hộp đen của các mô hình học máy.
- Khu vực tương tác và hậu xử lý kết quả: Kết quả cuối cùng không được hiển thị dưới dạng văn bản tĩnh mà được cấu trúc hóa thành các thẻ câu hỏi trắc nghiệm (MCQ Cards). Mỗi thẻ bao gồm câu hỏi, đáp án đúng và các phương án nhiễu được sắp xếp ngẫu nhiên để mô phỏng một bài thi thực tế. Tại đây, hệ thống tích hợp chức năng chỉnh sửa trực tiếp (Human-in-the-loop), cho phép người dùng hiệu chỉnh câu chữ hoặc thay đổi các lựa chọn nếu kết quả từ AI chưa đạt yêu cầu chuyên môn. Cuối cùng, chức năng xuất dữ liệu được thiết kế thuận tiện với các nút bấm tải xuống, hỗ trợ các định dạng phổ biến như CSV hoặc JSON, giúp giáo viên dễ dàng lưu trữ và tích hợp bộ đề vào các nền tảng quản lý học tập khác.

Tổng thể giao diện được tối ưu hóa về mặt bố cục và màu sắc để giảm mỏi mắt, hỗ trợ người dùng làm việc hiệu quả trong thời gian dài.

CHƯƠNG 3: CÀI ĐẶT CHƯƠNG TRÌNH

3.1. Cài đặt hệ thống và triển khai hệ thống

3.1.1. Môi trường cài đặt và cấu trúc thư mục

Link git dự án: phamvanso/Chuy-n-HTTT

Để hệ thống vận hành ổn định, dự án được triển khai trên môi trường Python 3.9+ với các thư viện lõi về học máy. Việc quản lý thư mục được tổ chức theo dạng mô-đun hóa để dễ dàng bảo trì và nâng cấp từng phần của pipeline.

Cấu trúc thư mục dự án:

- Thư mục gốc và các tệp tin thực thi chính:
  - train.py & evaluation.py: Các tệp tin kịch bản dùng để chạy quá trình huấn luyện mô hình và đánh giá tổng thể hệ thống trên các tập dữ liệu lớn.
  - requirements.txt: Chứa danh sách các thư viện phụ thuộc (transformers, ollama, streamlit, spacy, torch...).
- Thư mục demo_mcq/: Tập trung vào việc vận hành giao diện và điều phối các mô hình AI để phục vụ người dùng cuối.
  - app.py: Tệp tin khởi chạy giao diện Streamlit, quản lý trạng thái phiên làm việc.
  - generator.py: Chịu trách nhiệm nạp mô hình ViT5 và thực hiện các hàm sinh câu hỏi, đáp án.
  - distractor.py: Thực hiện kết nối và truyền tin với Ollama để tạo các phương án sai.
  - export_utils.py: Chứa các hàm hỗ trợ định dạng và xuất kết quả ra file phục vụ lưu trữ.
- Thư mục plms/: Đây là nơi chứa logic chuyên sâu về NLP.
  - spacy_module.py: Trái tim của giai đoạn tiền xử lý, thực hiện tách câu và nhận diện thực thể tên riêng.
  - language_model.py & inference_api.py: Các lớp đối tượng định nghĩa cấu trúc mô hình ngôn ngữ và giao diện lập trình ứng dụng nội bộ.

- model_evaluation_qag.py & compute_metrics.py: Cung cấp các hàm đo lường kỹ thuật như BLEU, ROUGE, METEOR để đánh giá độ tương đồng của văn bản sinh ra.
- trainer.py & data.py: Các module hỗ trợ việc quản lý tập dữ liệu và thực hiện vòng lặp huấn luyện mô hình.
- exceptions.py & utils.py: Chứa các hàm tiện ích và xử lý lỗi hệ thống để đảm bảo pipeline vận hành mượt mà.

- Thư mục llm/**:**
  - generate.py: Module chuyên biệt để tương tác với các mô hình ngôn ngữ lớn (LLM) cho các tác vụ sinh nội dung phức tạp.
  - trainer.py: Chứa logic nếu người dùng muốn tinh chỉnh (fine-tuning) thêm cho các dòng mô hình lớn.

3.1.2. Triển khai mô hình ViT5-QG-AE

Giai đoạn này hiện thực hóa khả năng cốt lõi của hệ thống: chuyển đổi văn bản thô thành cặp Câu hỏi và Đáp án đúng. Hệ thống sử dụng mô hình *shnl/vit5-vinewsqa-qg-ae*, một phiên bản kiến trúc Transformer được tối ưu hóa riêng cho tiếng Việt, giúp khắc phục các hạn chế về mặt ngữ pháp và ngữ nghĩa mà các mô hình đa ngôn ngữ thường gặp phải.

Cơ chế nạp và cấu hình mô hình: Trong tệp generator.py, mô hình được triển khai thông qua thư viện transformers của Hugging Face. Thay vì nạp thủ công, hệ thống sử dụng các lớp tự động hóa giúp đồng bộ hóa giữa bộ mã hóa văn bản và trọng số mô hình. Quá trình này đảm bảo rằng các token tiếng Việt được phân tách chính xác theo đặc thù ngôn ngữ đơn lập, tránh việc bị chia cắt thành các mảnh vô nghĩa.

Quy trình xử lý dữ liệu đầu vào: Để mô hình hiểu được tác vụ cần thực hiện là "Sinh câu hỏi và đáp án", dữ liệu đầu vào không được đưa trực tiếp vào mà phải được cấu trúc hóa theo định dạng mã lệnh. Hệ thống thêm vào một tiền tố định danh (Prefix), ví dụ: "vietnews_qg_ae: {context}". Tiền tố này đóng vai trò là tín hiệu để lớp Encoder của ViT5 kích hoạt vùng không gian đặc trưng đã được huấn luyện cho nhiệm vụ trích xuất câu hỏi và đáp án từ đoạn văn bản cho trước.

Kỹ thuật giải mã: Hiệu suất của việc sinh câu hỏi phụ thuộc rất lớn vào tham số giải mã tại tầng Decoder. Trong mã nguồn generator.py, hệ thống áp dụng kỹ thuật Beam Search với thông số num_beams=4. Thay vì chọn từ có xác suất cao nhất một cách máy móc (Greedy Search), Beam Search sẽ duy trì 4 nhánh tiềm năng nhất trong quá trình sinh, từ đó chọn ra chuỗi văn bản có tính mạch lạc và đúng cú pháp nhất. Các tham số như max_length được thiết lập ở mức 128 - 256 tokens để đảm bảo câu hỏi và đáp án sinh ra đủ ngắn gọn, súc tích nhưng vẫn chứa đựng đầy đủ ý nghĩa cần thiết.

Quản lý bộ nhớ và hiệu suất: Do các mô hình Transformer tiêu tốn khá nhiều tài nguyên GPU/RAM, tệp generator.py được thiết kế để nạp mô hình vào bộ nhớ đệm (Cache) ngay lần đầu khởi chạy. Điều này giúp các yêu cầu sinh câu hỏi tiếp theo được phản hồi gần như tức thì, không mất thời gian nạp lại mô hình. Kết quả đầu ra sau khi giải mã sẽ được làm sạch các token đặc biệt (skip_special_tokens=True) và chuyển thành định dạng Dictionary để sẵn sàng cung cấp dữ liệu cho module distractor.py xử lý ở bước kế tiếp.

3.1.3. Tích hợp Ollama cho distractor generation

Sau khi ViT5 hoàn thành việc sinh cặp Câu hỏi - Đáp án, hệ thống chuyển giao dữ liệu cho module demo_mcq/distractor.py. Nhiệm vụ của module này là kết nối với nền tảng Ollama đang chạy cục bộ để thực hiện tác vụ sinh phương án nhiễu (Distractor Generation). Việc tích hợp này được thực hiện thông qua giao thức API nội bộ, giúp tận dụng sức mạnh của các mô hình ngôn ngữ lớn (LLM) mà vẫn đảm bảo tính riêng tư của dữ liệu.

Cơ chế kết nối và điều phối: Trong file distractor.py, hệ thống khởi tạo một client kết nối tới server Ollama thông qua thư viện ollama-python. Thay vì xử lý đơn lẻ, module này được thiết kế để tiếp nhận đầu vào là các cặp (Q, A) từ mô hình ViT5. Điểm then chốt tại đây là việc cấu trúc hóa tham số đầu vào (Prompting). Hệ thống sử dụng kỹ thuật Few-shot hoặc Zero-shot Prompting tùy theo cấu hình, trong đó chỉ thị mô hình phải bám sát câu hỏi và đáp án chính xác để tạo ra đúng 03 phương án sai mang tính logic cao.

Xử lý và chuẩn hóa dữ liệu đầu ra: Một thách thức lớn khi làm việc với LLM là tính bất định của văn bản trả về. Để giải quyết vấn đề này, distractor.py tích hợp các hàm hậu xử lý (post-processing) mạnh mẽ. Dữ liệu từ Ollama trả về dưới dạng chuỗi văn bản thô sẽ được đưa qua các bộ lọc để:

- Tách chuỗi: Sử dụng các ký tự phân tách (như dấu phẩy hoặc xuống dòng) để bóc tách thành một danh sách chứa chính xác 3 phương án.
- Làm sạch dữ liệu: Loại bỏ các phần giải thích dư thừa, các ký tự đặc biệt hoặc số thứ tự không cần thiết mà LLM thường tự ý thêm vào.
- Kiểm soát số lượng: Đảm bảo hệ thống luôn nhận đủ 3 phương án nhiễu trước khi chuyển dữ liệu sang giao diện hiển thị.

Tối ưu hóa tham số thực thi: Thiết lập temperature ở mức vừa phải trong mã nguồn giúp đảm bảo các phương án nhiễu không bị lặp lại nhưng cũng không quá xa rời chủ đề của câu hỏi. Điều này tạo ra một sự cân bằng giữa tính đa dạng và tính chính xác về mặt ngữ nghĩa, giúp bộ đề thi trắc nghiệm trở nên chuyên nghiệp và có độ khó phù hợp.

3.1.4. Xây dựng giao diện Streamlit

Tệp demo_mcq/app.py đóng vai trò là tầng giao tiếp trực tiếp với người dùng và là "trạm điều phối" kết nối tất cả các module xử lý phía sau. Sử dụng framework Streamlit, giao diện được thiết kế để chuyển đổi các luồng xử lý dữ liệu phức tạp thành một trải nghiệm tương tác đơn giản, tập trung vào tính ứng dụng thực tế cho giáo viên và người biên soạn đề thi.

Quản lý luồng công việc: Một điểm quan trọng trong app.py là việc sử dụng st.session_state để duy trì dữ liệu qua các lần tương tác. Khi người dùng nhấn nút "Sinh đề thi", hệ thống sẽ lưu trữ danh sách các câu hỏi, đáp án và phương án nhiễu vào bộ nhớ đệm của phiên làm việc. Điều này cho phép người dùng có thể chỉnh sửa nội dung từng câu hỏi hoặc thay đổi các tùy chọn hiển thị mà không làm mất đi các kết quả đã sinh ra từ các mô hình ViT5 và Ollama trước đó.

Thiết kế các phân vùng chức năng:

- Bảng điều khiển nhập liệu: Sử dụng st.text_area để tiếp nhận văn bản ngữ cảnh. Hệ thống được cấu hình để kiểm soát độ dài đầu vào, đảm bảo dữ liệu truyền vào module generator.py nằm trong giới hạn xử lý của mô hình ViT5.
- Khu vực hiển thị danh sách câu hỏi: Sau khi quá trình inference hoàn tất, dữ liệu được trình bày dưới dạng các thẻ câu hỏi trắc nghiệm. Tại đây, app.py thực hiện logic xáo trộn vị trí của đáp án đúng trong danh sách 4 phương án (A, B, C, D) bằng thư viện random, đảm bảo tính khách quan cho bộ đề thi.
- Tính năng hiệu chỉnh (Human-in-the-loop): Giao diện cung cấp các ô nhập liệu cho phép người dùng chỉnh sửa trực tiếp nội dung câu hỏi hoặc các phương án nhiễu ngay trên màn hình nếu cảm thấy kết quả của AI chưa thực sự tối ưu.

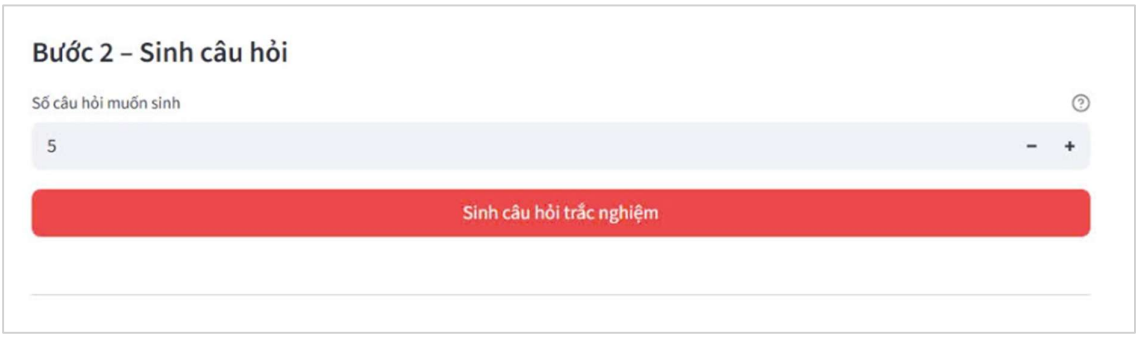
Tích hợp công cụ xuất bản: Cuối cùng, app.py kết nối với module export_utils.py để hiện thực hóa chức năng lưu trữ. Người dùng có thể chọn định dạng đầu ra (như CSV hoặc JSON) thông qua các nút bấm st.download_button. Luồng dữ liệu lúc này sẽ được định dạng lại từ cấu trúc Dictionary trong bộ nhớ thành tệp tin có cấu trúc chuẩn, giúp người dùng dễ dàng đưa vào các hệ thống quản lý học tập (LMS) hoặc in ấn phục vụ kiểm tra.

3.2. Kết quả cài đặt

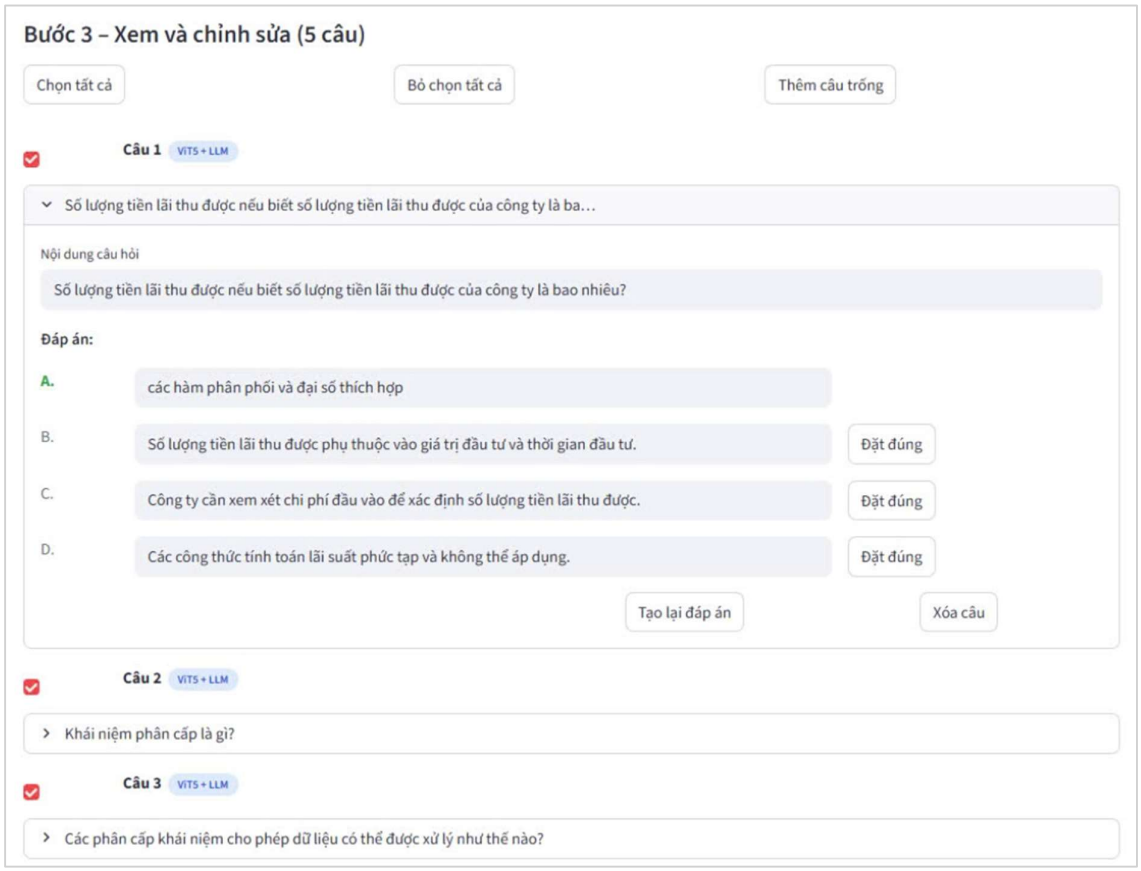
Sau khi hoàn thiện việc kết nối các module từ xử lý ngôn ngữ đến giao diện người dùng, hệ thống đã được vận hành thử nghiệm trên các tập dữ liệu tin tức và văn bản giáo khoa tiếng Việt. Quá trình đánh giá này tập trung vào khả năng phản hồi thời gian thực và chất lượng của bộ đề trắc nghiệm được sinh ra.

Giao diện của hệ thống được tối ưu hóa để đảm bảo tính trực quan. Dưới đây là hình ảnh thực tế của ứng dụng khi vận hành, minh họa cho quy trình từ khi nhập văn bản thô đến khi nhận được bộ câu hỏi hoàn chỉnh:

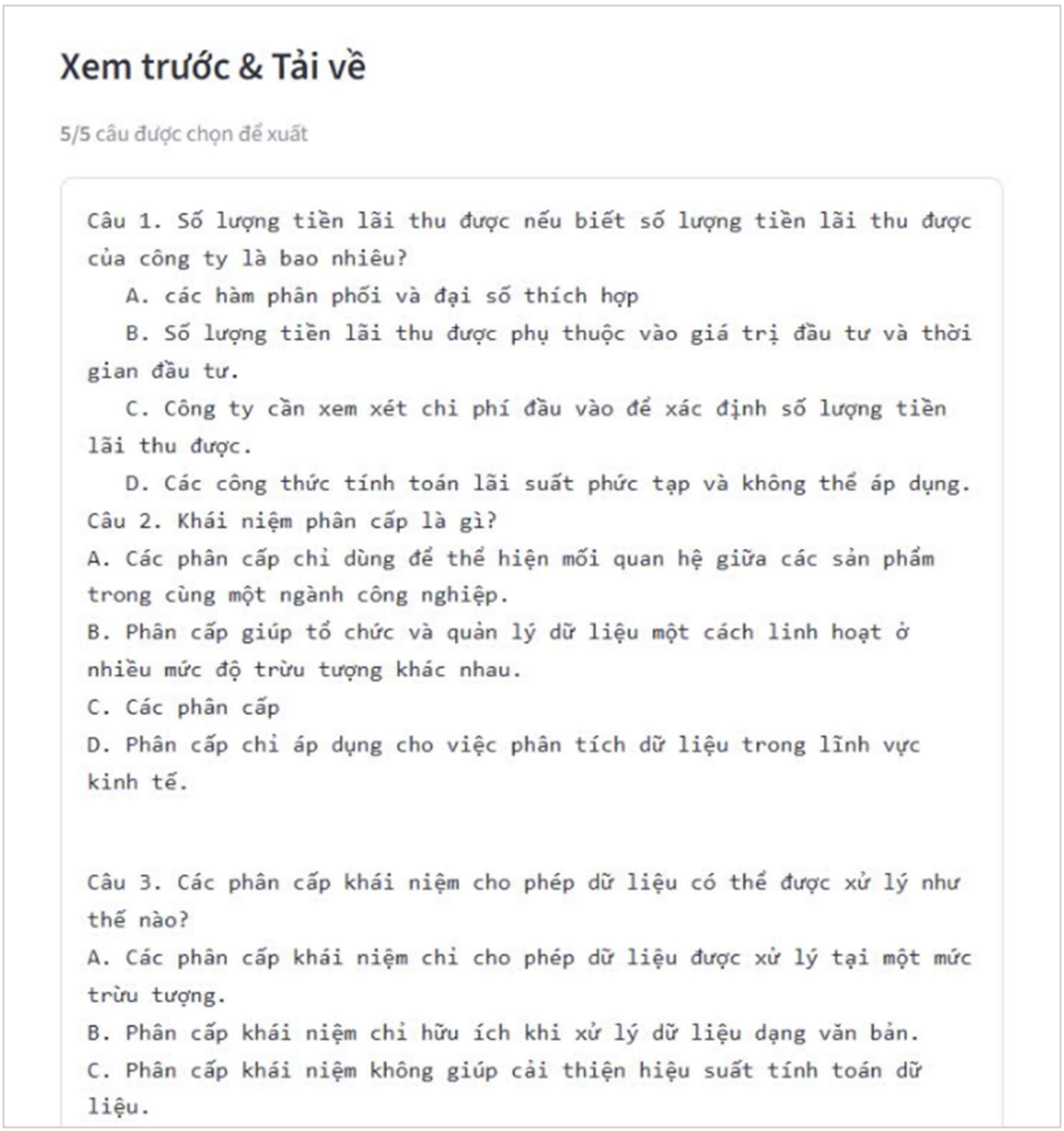
Hình 2: Giao diện khi người dùng nhập văn bản đầu vào



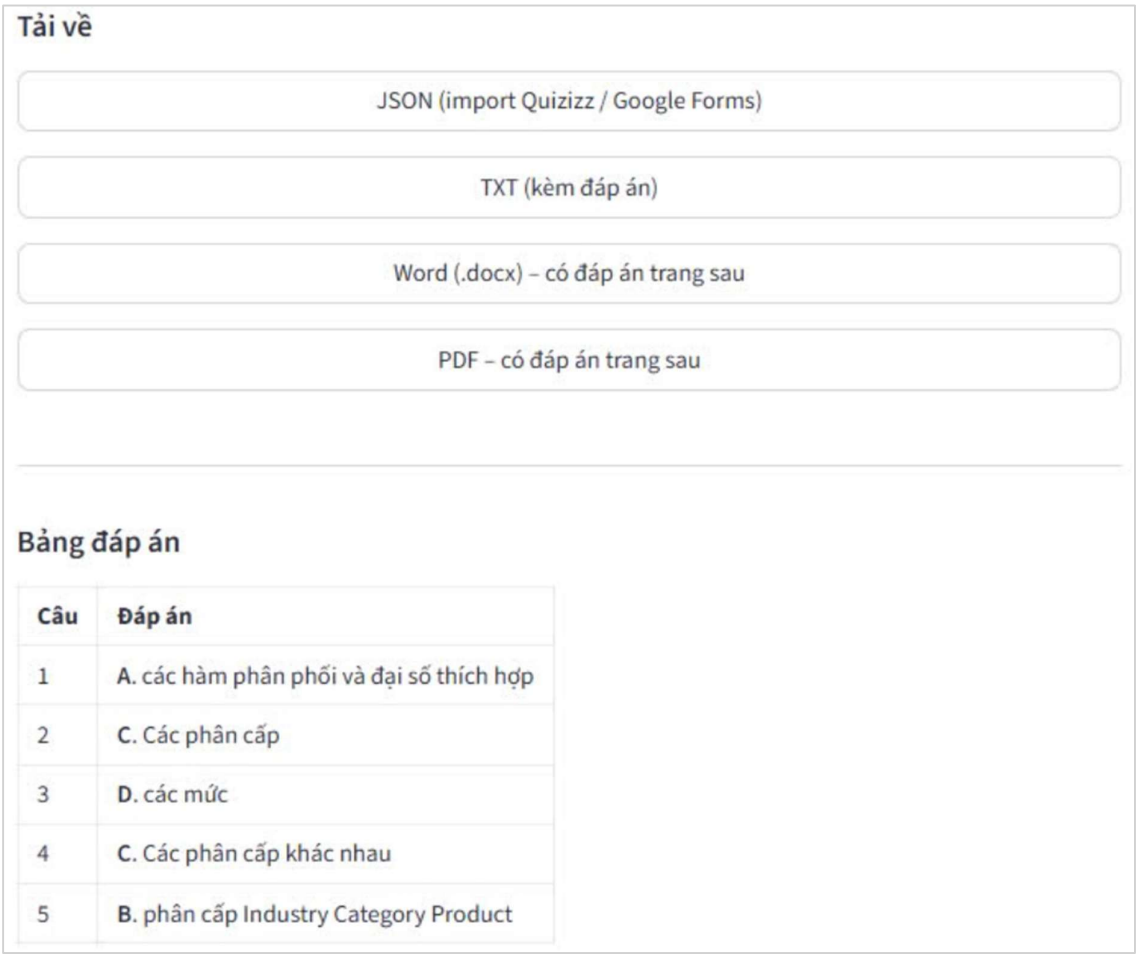
Hình 3: Giao diện chọn số câu hỏi muốn sinh



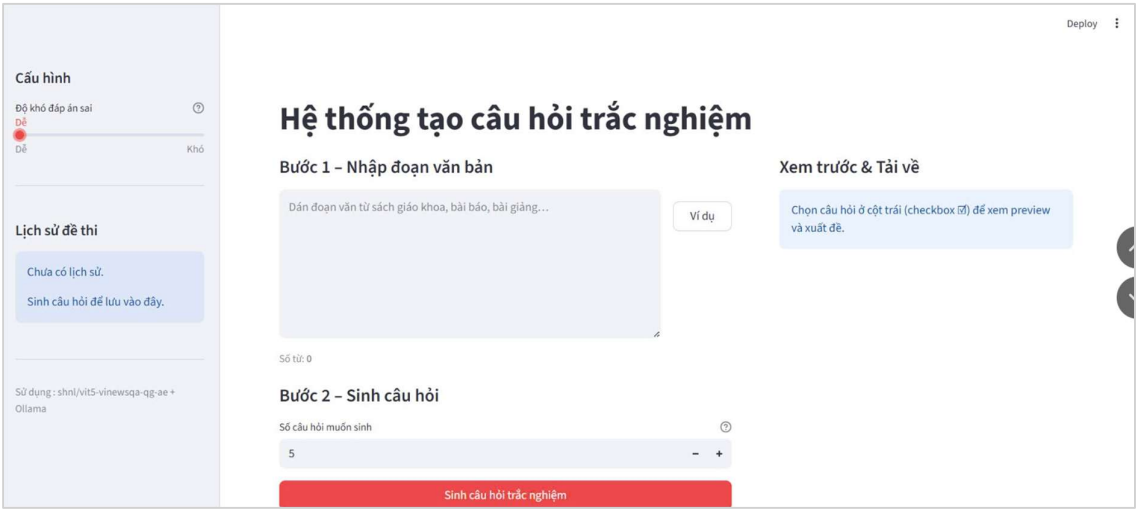
Hình 4: Giao diện kết quả sinh và chỉnh sửa, xáo trộn đáp án của người dùng



Hình 5: Giao diện xem trước khi người dùng tải file câu hỏi về



Hình 6: Giao diện chọn loại file tải và bảng đáp án



Hình 7: Giao diện chọn độ khó và xem lịch sử tạo

3.3. Kết luận

Về khả năng sinh câu hỏi (ViT5): Qua thực nghiệm, mô hình *shnl/vit5-vinewsqa-qg-ae* cho thấy khả năng trích xuất thông tin rất chính xác. Các câu hỏi tập trung mạnh vào các thực thể như mốc thời gian, địa điểm và nhân vật quan trọng. Cấu trúc câu hỏi sinh ra tự nhiên, không bị lặp từ và bám sát ngữ cảnh của đoạn văn đầu vào.

Về chất lượng phương án nhiễu (Ollama): Việc sử dụng LLM qua Ollama đã tạo ra sự khác biệt rõ rệt so với các phương pháp sinh từ ngẫu nhiên truyền thống. Các phương án sai có sự tương đồng cao về mặt ngữ nghĩa với đáp án đúng, tạo ra độ nhiễu cần thiết. Đặc biệt, nhờ kỹ thuật Prompting trong distractor.py, các phương án nhiễu luôn đảm bảo cùng loại thực thể (ví dụ: cùng là các thành phố nếu đáp án đúng là một thành phố), giúp nâng cao giá trị sư phạm của câu hỏi.

Về hiệu suất vận hành: Nhờ cơ chế quản lý trạng thái trong app.py, hệ thống vận hành mượt mà. Thời gian để sinh một bộ 5-10 câu hỏi trắc nghiệm hoàn chỉnh dao động từ 20 đến 40 giây (tùy thuộc vào cấu hình phần cứng chạy Ollama cục bộ). Việc tích hợp các thông báo trạng thái đã giúp cải thiện đáng kể trải nghiệm người dùng trong thời gian chờ đợi mô hình suy luận.

PHẦN KẾT LUẬN

1. Tóm tắt các nội dung đã thực hiện

Dự án đã nghiên cứu và xây dựng thành công hệ thống tạo câu hỏi trắc nghiệm tự động từ văn bản, một giải pháp tự động hóa quy trình tạo câu hỏi trắc nghiệm cho tiếng Việt. Các nội dung chính đã hoàn thành bao gồm:

- Xây dựng luồng xử lý hoàn chỉnh từ khâu tiếp nhận văn bản thô, tiền xử lý ngôn ngữ bằng SpaCy, đến khâu trích xuất tri thức.
- Triển khai thành công mô hình ViT5 chuyên biệt cho tác vụ sinh câu hỏi và đáp án, đảm bảo tính chính xác về mặt sự thật dựa trên văn bản gốc.
- Tích hợp linh hoạt mô hình ngôn ngữ lớn (LLM) thông qua Ollama để giải quyết bài toán sinh phương án nhiễu bằng kỹ thuật Prompt Engineering.
- Thiết kế và cài đặt giao diện người dùng trực quan bằng Streamlit, hỗ trợ đầy đủ các tính năng từ chỉnh sửa nội dung đến xuất bản dữ liệu đa định dạng (CSV, JSON).

2. Những điểm đạt được và hạn chế của hệ thống

Những điểm đạt được:

- Tính thực tiễn cao: Hệ thống rút ngắn đáng kể thời gian biên soạn học liệu, giúp giáo viên tạo ra bộ đề thi chỉ trong vài giây thay vì hàng giờ làm việc thủ công.
- Chất lượng câu hỏi: Nhờ sự kết hợp giữa ViT5 và LLM, câu hỏi sinh ra đạt độ tự nhiên về ngôn ngữ và các phương án nhiễu có tính logic cao, bám sát thực thể của đáp án đúng.
- Kiến trúc linh hoạt: Việc mô-đun hóa mã nguồn giúp hệ thống dễ dàng thay đổi hoặc nâng cấp các mô hình ngôn ngữ khác nhau mà không ảnh hưởng đến toàn bộ quy trình.

Những hạn chế:

- Phụ thuộc phần cứng: Do chạy các mô hình Deep Learning và LLM cục bộ, tốc độ xử lý vẫn phụ thuộc nhiều vào cấu hình GPU/RAM của máy tính triển khai.

- Khả năng xử lý ngữ cảnh cực dài: Với các văn bản quá dài hoặc quá phức tạp, mô hình đôi khi vẫn gặp khó khăn trong việc bao quát toàn bộ ý chính để đặt câu hỏi mang tính tổng hợp cao.

3. Hướng phát triển trong tương lai

Để hoàn thiện hệ thống, các hướng phát triển tiếp theo sẽ tập trung vào đầu tiên là nâng cao chất lượng mô hình, tiến hành tinh chỉnh (fine-tuning) sâu hơn mô hình LLM trên các bộ dữ liệu chuyên ngành (Y học, Luật, Kỹ thuật) để tạo ra các phương án nhiễu có độ lắt léo và chuyên môn sâu hơn. Thứ hai sẽ đa dạng hóa đầu vào, tích hợp thêm công nghệ OCR để hệ thống có thể đọc dữ liệu trực tiếp từ ảnh chụp sách giáo khoa hoặc các tệp tin PDF định dạng phức tạp. Tiếp theo là đánh giá độ khó tự động, phát triển thuật toán phân loại độ khó của câu hỏi dựa trên mức độ tư duy (theo thang Bloom), giúp giáo viên tự động thiết lập ma trận đề thi cân đối. Cuối cùng là triển khai Cloud, chuyển đổi hệ thống lên các nền tảng điện toán đám mây để người dùng có thể truy cập và sử dụng mọi lúc, mọi nơi mà không lo ngại về giới hạn phần cứng cá nhân.

TÀI LIỆU THAM KHẢO

[1] Pham, Q.-H., Le, H.-L., Dang, N. M., Tran, K. T., Tran-Tien, M., Dang, V.-H., Vu, H.-T., Nguyen, M.-T., & Phan, X.-H. (2024). *Towards Vietnamese Question and Answer Generation: An Empirical Study.* ACM Transactions on Asian and Low-Resource Language Information Processing. https://doi.org/10.1145/3675781

[2] Phan, L. T., Tran, H., Nguyen, H., & Trinh, T. H. (2022). *ViT5: Pretrained Text-to-Text Transformer for Vietnamese Language Generation.* Proceedings of NAACL-HLT 2022 (Student Research Workshop). https://aclanthology.org/2022.naacl-srw.18

[3] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* Journal of Machine Learning Research, 21(140), 1–67. https://jmlr.org/papers/v21/20-1307.html

[4] Nguyen, K., Nguyen, V. D., Nguyen, A. G.-T., & Nguyen, N. L.-T. (2020). *A Vietnamese Dataset for Evaluating Machine Reading Comprehension (ViQuAD).* Proceedings of COLING 2020. https://aclanthology.org/2020.coling-main.233

[5] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., … Brew, J. (2020). *Transformers: State-of-the-Art Natural Language Processing.* Proceedings of EMNLP 2020 (System Demonstrations). https://aclanthology.org/2020.emnlp-demos.6

[6] Google DeepMind. (2024). *Gemini: A Family of Highly Capable Multimodal Models.* https://deepmind.google/technologies/gemini/

[7] Streamlit Inc. (2024). *Streamlit – A faster way to build and share data apps.* https://streamlit.io

[8] ViQAG original codebase: https://github.com/Shaun-le/ViQAG

[9] shnl/vit5-vinewsqa-qg-ae model: https://huggingface.co/shnl/vit5-vinewsqa-qg-ae